Assignment 2

Sannan Shakeel (23i-3017)
WEB-A

1. Game Logic and Working Mechanism

1.1 Match Structure

Cricket Bash is a single-player 2D batting game built with React (Vite). Each match simulates a short-format innings governed by fixed constraints: the player faces exactly 2 overs (12 balls total) and has 2 wickets. The innings concludes as soon as either constraint is exhausted — whichever comes first.

Match Constraints at a Glance

Total balls per match: 12 (2 overs of 6 balls each)
Maximum wickets before game over: 2
Possible run outcomes per ball: 0, 1, 2, 3, 4, 6
Special outcome: W (Wicket) — no runs scored, wicket count incremented

1.2 State Machine

The game flows through a strict sequence of phases managed entirely in React state. No phase can be skipped or overlapped. The animationPhase field drives both the UI rendering and the animation timing pipeline.

- waiting — Slider is active. Player selects batting style and clicks Play Shot.
- bowling — Shot confirmed. Ball travel and bat-swing animations are running.
- showing — Animation complete. Outcome applied to score. Commentary displayed.
- Game Over — Triggered when wickets >= 2 or ballsBowled >= 12.

1.3 Per-Ball Flow

Every single delivery follows this exact sequence:

Step	Action
1	Player selects Aggressive or Defensive batting style
2	Slider is moving continuously across the power bar (requestAnimationFrame loop)

Step	Action
3	Player clicks Play Shot — slider.freeze() is called, position (0–1) is captured
4	Outcome is determined deterministically from slider position and segment map (no Math.random)
5	Bowling animation starts: ball travels from bowler to batsman over 0.8s
6	At 880ms: bat-swing animation triggers, impact flash appears
7	At 1400ms: applyOutcome() updates runs, wickets, ballsBowled in state
8	Commentary line displayed; scoreboard updates with animation
9	At 2800ms: phase resets to waiting — slider resumes for next ball

1.4 State Management

All game state lives in a single custom hook (useGameState) using React's useState. The relevant fields are:

- runs — cumulative run total for the innings
- wickets — number of wickets fallen (max 2)
- ballsBowled — delivery count 0–12
- style — current batting style: 'aggressive' | 'defensive'
- animationPhase — controls rendering and timing: 'waiting' | 'bowling' | 'showing'
- lastOutcome — most recent delivery result ('W', '0', '1', '2', '3', '4', '6')
- isGameOver — boolean; true when wickets or balls threshold is reached

1.5 Commentary System

After each delivery, a commentary line is drawn from a pool of 5 predefined strings per outcome (7 outcomes × 5 lines = 35 total lines). Math.random() is used exclusively here as a flavour-text selector — it has zero effect on the shot outcome itself.

2. Mapping of Probabilities to the Power Bar

2.1 The Probability Distributions

Two probability distributions are defined in gameData.js — one per batting style. Each distribution assigns a weight to every possible outcome. The weights must, and do, sum to exactly 1.0. The design reflects realistic cricket risk profiles: Aggressive batting yields more boundaries but higher dismissal risk, while Defensive batting prioritises run accumulation with low wicket probability.

Outcome	Aggressive	Defensive	Segment Color
Wicket (W)	15%	5%	Danger Red #C0392B
Dot (0)	8%	20%	Steel Gray #555E6B
1 Run	12%	35%	Sky Blue #3498DB
2 Runs	10%	22%	Green #27AE60
3 Runs	5%	8%	Purple #8E44AD
Four (4)	28%	7%	Orange #E67E22
Six (6)	22%	3%	Gold #F1C40F
TOTAL	100%	100%	—

2.2 Segment Construction

The buildSegments() function in gameData.js converts a probability distribution into an ordered array of segment objects. Each segment stores its outcome label, its start and end positions on the 0–1 number line, its colour, and its probability. Segments are arranged in a fixed order: W, 0, 1, 2, 3, 4, 6.

Example — Aggressive distribution segment boundaries:

```
W    : start=0.00, end=0.15  (width 15%)
0    : start=0.15, end=0.23  (width  8%)
1    : start=0.23, end=0.35  (width 12%)
2    : start=0.35, end=0.45  (width 10%)
3    : start=0.45, end=0.50  (width  5%)
4    : start=0.50, end=0.78  (width 28%)
6    : start=0.78, end=1.00  (width 22%)
```

2.3 Visual Rendering on the Bar

The PowerBar component renders one absolutely-positioned div per segment inside a relative container. Each segment's left CSS value equals segment.start × 100% and its width equals segment.probability × 100%. This ensures pixel-perfect proportional representation — a 28% segment occupies exactly 28% of the bar's width.

Rendering Formula

left = segment.start × 100% (e.g. 0.50 → 50%)
width = segment.probability × 100% (e.g. 0.28 → 28%)
Each segment renders its outcome label (W, DOT, 1 RUN, FOUR!, SIX!) centred inside.
When a shot is played, the segment containing the frozen position is highlighted with increased brightness.

2.4 Outcome Resolution — No Randomness

This is the critical rule of the game: the outcome of every delivery is determined solely by the position of the slider when the player clicks Play Shot. The getOutcomeFromPosition() function performs a simple linear scan of the segment array:

```
function getOutcomeFromPosition(position, segments) {
  const seg = segments.find(
    (s) => position >= s.start && position < s.end
  );
  return seg ? seg.outcome : segments[segments.length - 1].outcome;
}
```

Math.random() is never called in this path. The player can theoretically achieve any outcome by stopping the slider at the precise position on the bar. The probability distribution simply determines how wide each zone is — a wider zone is easier to land in.

2.5 Style Switching

The player can switch between Aggressive and Defensive before each delivery (while the slider is in the waiting phase). Switching calls buildSegments() with the new style, which rebuilds the segment array and re-renders the bar with the new colour widths instantly. The slider position reference (posRef) is unaffected — it continues from wherever it is.

3. Implementation of Animations

3.1 Slider Animation (Power Bar)

The slider is the most performance-critical animation in the game. It must run at 60fps continuously without causing React re-renders, which would cause the entire component tree to reconcile on every frame.

The solution uses three techniques together:

- **requestAnimationFrame:** requestAnimationFrame loop — The animate() function calls itself recursively via rAF. This synchronises the animation to the display refresh rate (typically 60Hz) and pauses automatically when the tab is hidden.
- **Direct DOM mutation:** Direct DOM manipulation — The thumb element is moved by setting thumbRef.current.style.left directly on the real DOM node, completely bypassing React's virtual DOM diffing. No setState is called per frame.
- **useRef for animation state:** useRef for mutable state — posRef (slider position), dirRef (direction), and frozenRef (freeze flag) are stored in useRef containers. Mutating refs does not trigger re-renders.

Slider Loop Pseudocode

```
const animate = () => {
  if (frozenRef.current) return;           // stopped by freeze()
  posRef.current += SPEED * dirRef.current; // advance position
  if (pos >= 1) dirRef.current = -1;       // bounce at right edge
  if (pos <= 0) dirRef.current = +1;       // bounce at left edge
  thumbRef.current.style.left = pos*100%+;  // move DOM directly
  rafRef.current = requestAnimationFrame(animate); // next frame
};
```

Speed is set to 0.009 position units per frame, giving a full left-to-right sweep in approximately 1.1 seconds at 60fps. The slider bounces (ping-pong) between 0 and 1 continuously.

3.2 Freeze Mechanism

When the player clicks Play Shot, the PowerBar's imperative freeze() method is invoked via a React ref (useImperativeHandle). freeze() sets frozenRef.current = true, which causes the animate loop to exit on the next frame. The final posRef.current value is captured and returned to the Game component, which passes it to getOutcomeFromPosition() for deterministic outcome resolution.

3.3 Ball Travel Animation

The ball animation is implemented as a CSS class-swap state machine. The ball element is absolutely positioned and transitions between four CSS classes, each with a different right value. React state (ballPhase) controls which class is applied. Transitions are declared purely in CSS.

Phase	CSS Class	Behaviour
reset	.ball-phase-reset	Ball placed at bowler position (right: 12%), invisible (opacity: 0)
travel	.ball-phase-travel	CSS transition: right moves from 12% to 72% over 0.8s with cubic-bezier easing
hit	.ball-phase-hit	Ball frozen at right: 72% (batsman contact point), transition disabled
fly	.ball-phase-fly	transform: translate(x, y) moves ball outward; right: 72% anchors start

A critical implementation note: the ball-phase-fly class explicitly re-declares right: 72% alongside its transform. Without this, when React removes ball-phase-hit and applies ball-phase-fly, the right property reverts to the browser default (typically 0 or auto), causing the ball to snap to the far left before the translate animation runs. Anchoring right in the fly class eliminates this jump entirely.

3.4 Bat Swing Animation

The bat-arm element is positioned on the right side of the batsman figure using right: -14px, reflecting the correct stance (batsman faces the bowler on the right). The swing animation is a CSS @keyframes sequence triggered by adding the swing-up class:

```
@keyframes batSwingAnim {
  0%   { transform: rotate(20deg);   }  /* resting stance */
  40%  { transform: rotate(120deg);  }  /* back-lift      */
  70%  { transform: rotate(-30deg);  }  /* follow-through */
  100% { transform: rotate(20deg);   }  /* return to rest */
}
```

The class is applied at the 880ms mark inside the bowling setTimeout sequence — precisely when the ball arrives at right: 72% — ensuring the bat visually contacts the ball at the correct moment.

3.5 Timing Orchestration

All animation phases are coordinated using a series of setTimeout calls inside a useEffect that watches animationPhase. When animationPhase becomes 'bowling', the following sequence fires:

- t=80ms — ballPhase set to 'travel'; CSS transition begins moving ball toward batsman
- t=880ms — ballPhase set to 'hit'; batSwing and showImpact set to true
- t=1050ms — ballPhase set to 'fly'; ball translates outward from contact point
- t=1100ms — showImpact cleared (flash disappears)
- t=1400ms — applyOutcome() called; game state updated, scoreboard animates
- t=2800ms — animationPhase reset to 'waiting'; slider resumes

All timeouts are cleared in the useEffect cleanup function, preventing state updates on unmounted components or after a restart.

3.6 Wicket Fall Animation

When the outcome is a wicket (W) and animationPhase transitions to 'showing', the fallen CSS class is conditionally applied to the batsman-wickets element. The class applies a CSS transform that topples the stumps to the left:

```
.wickets-set.fallen {
  transform: translateX(-25px) rotate(-40deg);
}
```

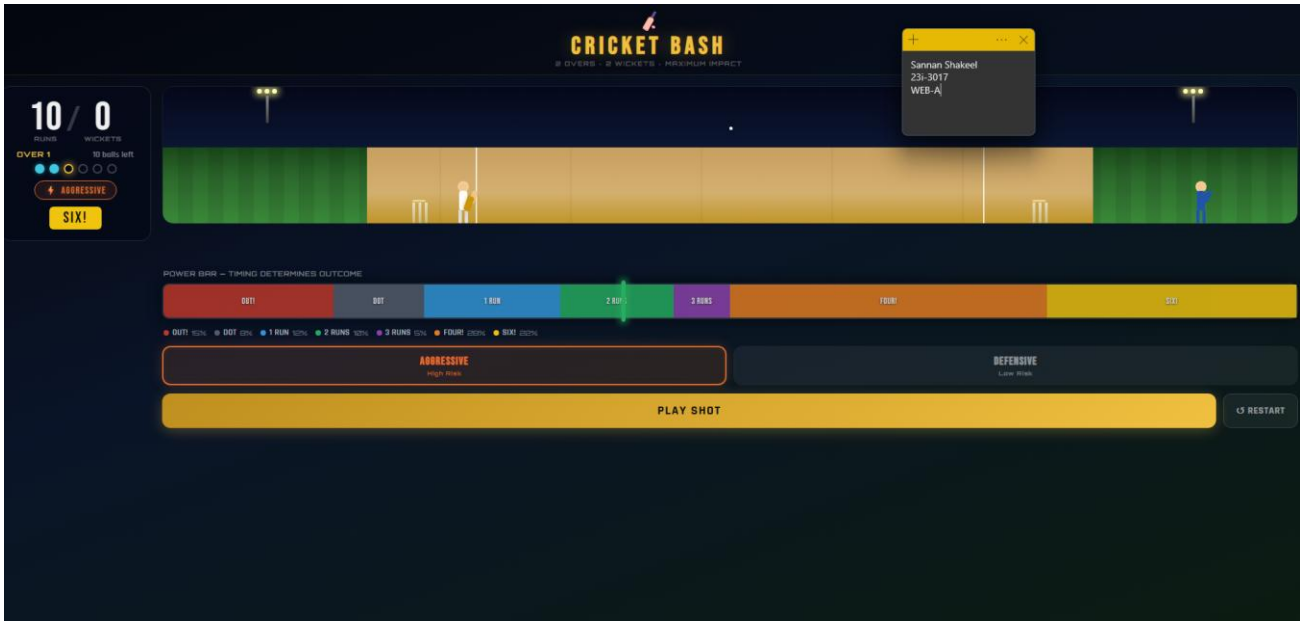
The negative X translation and negative rotation angle produce the correct visual — stumps flying backward (away from the bowler's direction of travel). A 0.4s ease transition on the wickets-set class smoothly interpolates from the upright to fallen state.

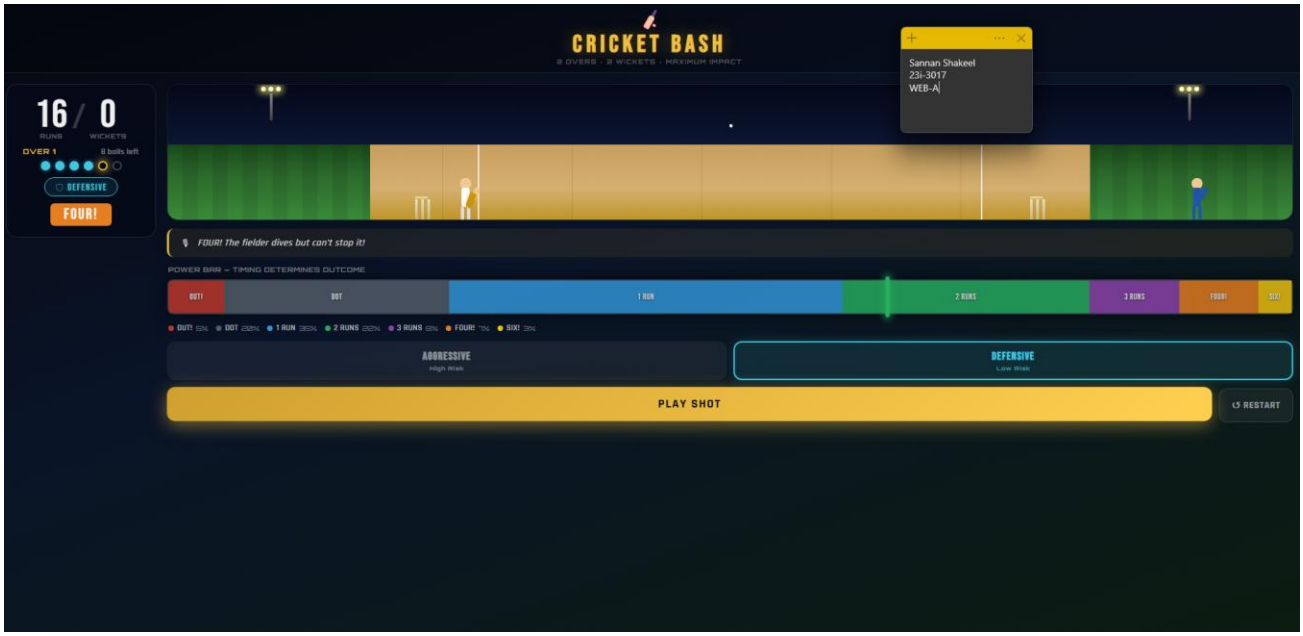
3.7 Score Flash and Commentary

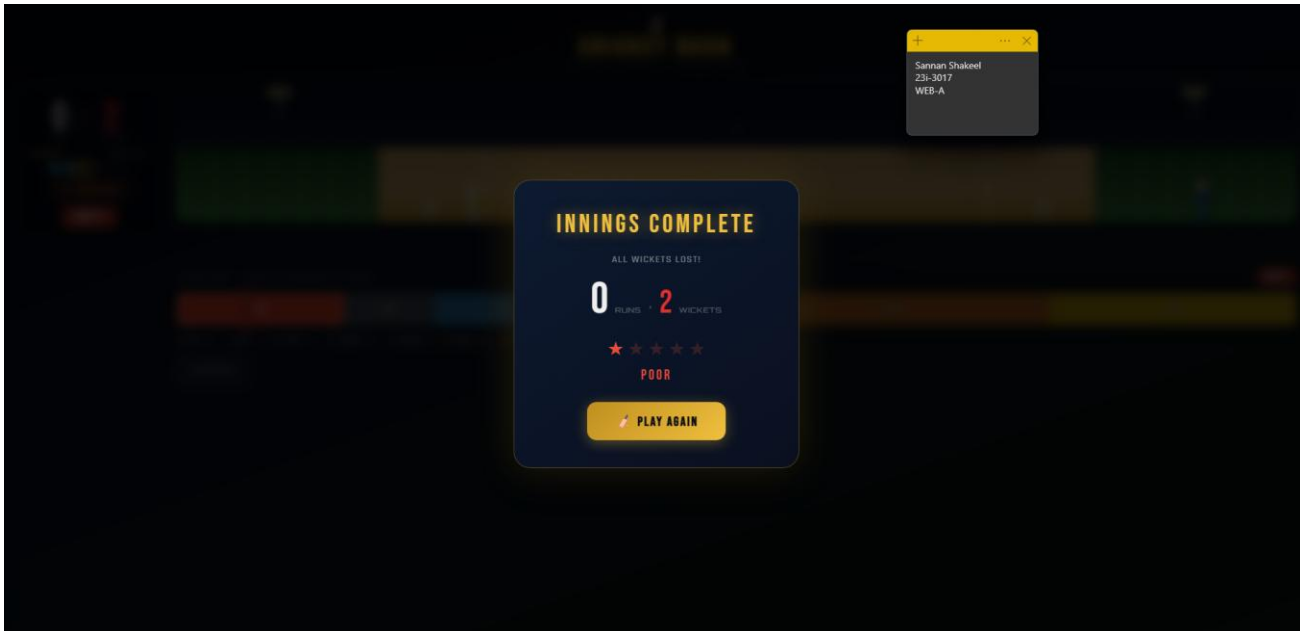
When runs are scored, a scoreFlash boolean is set in game state. The Scoreboard component watches this via useEffect and adds a score-flash CSS class to the runs element, triggering a keyframe that scales and recolours the number gold momentarily. The class is removed after 600ms and the flag is cleared, making it re-triggerable on each scoring delivery.

Commentary text is rendered in the Commentary component with a commentaryKey prop that increments on every ball. The key forces React to remount the component, restarting the CSS animation even when two consecutive deliveries produce the same outcome and the text string is identical.

Screenshots






Sannan Shakeel
23i-3017
WEB-A
INNINGS COMPLETE
ALL WICKETS LOST!
0 RUNS
2 WICKETS
POOR
PLAY AGAIN